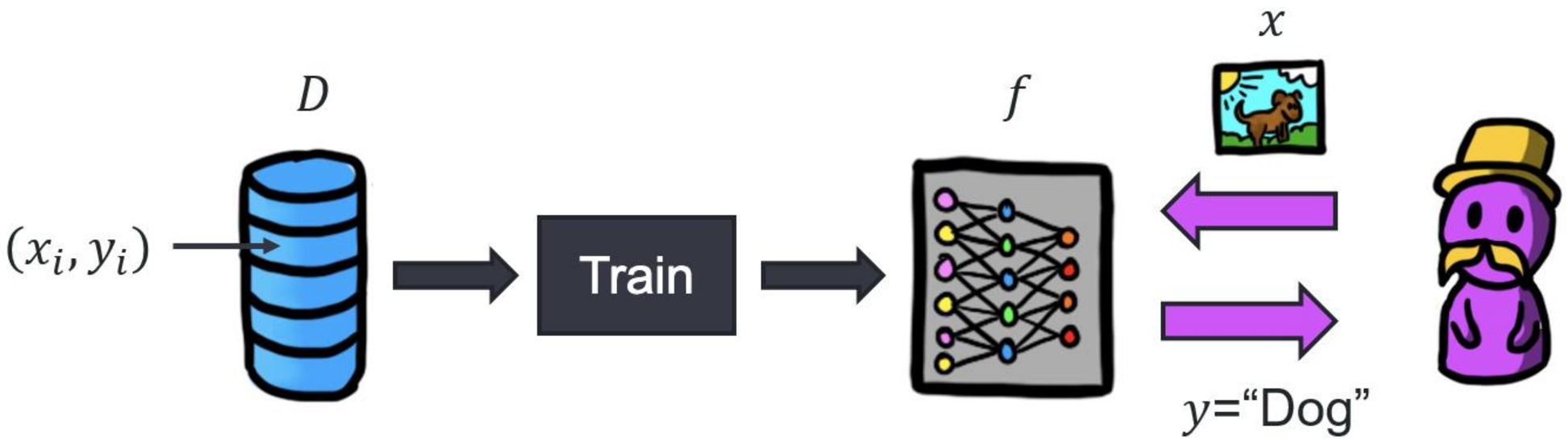
CS459/659
Privacy, Cryptography, Network and Data Security

Adversarial Machine Learning

Spring 2026, Tuesday/Thursday 10:00-11:20am

Machine Learning - Recap



Machine Learning - Recap

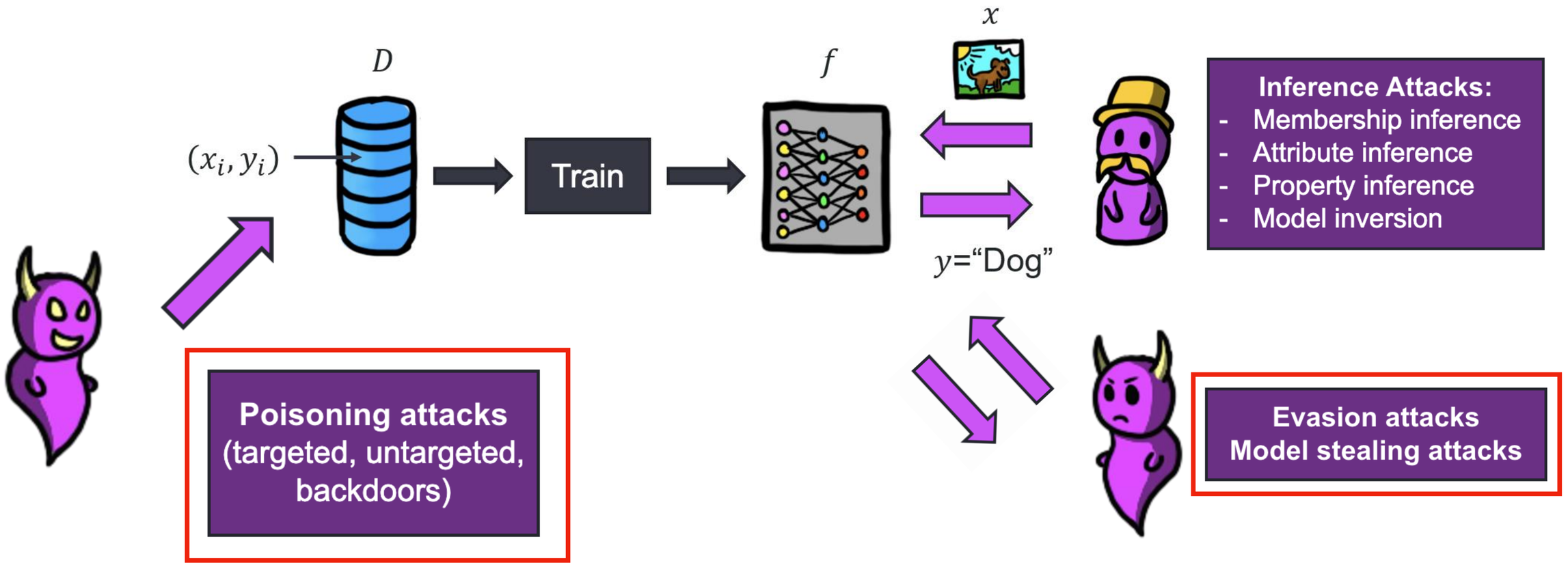
ML model is a learned, parametrized function. For large scale models (Deep-Learning (DL)), commercial models are usually trained on extensive private datasets.

There are three main forms of ML:

- Supervised: classification, tokenized generation methods (ChatGPT)
- Unsupervised: clustering, synthetic data generation
- Reinforcement Learning: games (Chess, Go, Poker…), robotics

Attacking Machine Learning

Machine Learning - Attacks recap



Part 1: Intellectual Property

Intellectual Property - Topics

- Machine Learning as a Service (MLaaS)
- Model Stealing
  - Introduction & Motivation
  - Attacks
  - Defenses
- IP protection
  - Watermarking

Machine Learning as a Service

- Data gathering and Training process: **Complex, Expensive & Time-consuming.**
  - In particular, for classification, labeling has to be done by humans

Machine Learning as a Service

- Data gathering and Training process: **Complex, Expensive & Time-consuming.**
  - In particular, for classification, labeling has to be done by humans

- Solution: Machine Learning-as-a-Service (MLaaS).
  - Offer model as a queryable black-box service (ChatGPT).
  - Requires significant computing capabilities to provide accessible service
  - If frequent queries are necessary, can become **quite expensive** for the user.

Machine Learning as a Service

- Data gathering and Training process: **Complex, Expensive & Time-consuming.**
  - In particular, for classification, labeling has to be done by humans

- Solution: Machine Learning-as-a-Service (MLaaS).
  - Offer model as a queryable black-box service (ChatGPT).
  - Requires significant computing capabilities to provide accessible service
  - If frequent queries are necessary, can become **quite expensive** for the user.



What if we just steal someone's else's MLaaS model?

Model Stealing

Model Stealing - What is there to steal?

- **Approximation of the behaviour of the model**

Model Stealing - What is there to steal?

- **Approximation of the behaviour of the model**
- **Model architecture**
- **Learned parameters**
- **Training hyper-parameters**

Model Stealing - Simple attack

Approximating the behaviour of the model:

- Let $f(x, \theta) = y$ represent the model we are trying to steal. It is a learned parametrized function f with parameters θ trained on a dataset $D = (X, Y)$.
- Assume we have some unlabeled auxiliary dataset $D' = (X', \cdot)$ that could be significantly smaller than D.

Model Stealing - Simple attack

Approximating the behaviour of the model:

- Let $f(x, \theta) = y$ represent the model we are trying to steal. It is a learned parametrized function f with parameters θ trained on a dataset $D = (X, Y)$.
- Assume we have some unlabeled auxiliary dataset $D' = (X', \cdot)$ that could be significantly smaller than D.

- We create our own model f' with parameters θ' and create labels for it as $f(X') = Y'$.

Model Stealing - Simple attack

Approximating the behaviour of the model:

- Let $\boldsymbol{f}(x, \theta) = y$ represent the model we are trying to steal. It is a learned parametrized function f with parameters θ trained on a dataset $D = (X, Y)$.
- Assume we have some unlabeled auxiliary dataset $D' = (X', \cdot)$ that could be significantly smaller than D.

- We create our own model f' with parameters θ' and create labels for it as $\boldsymbol{f}(X') = Y'$.

- We can now train our model with $D' = (X', Y')$.

Model Stealing - Literature

Information	Paper	Approach	Reducing Query	Recovery Rate (%) for Models					
				SVM	DT	LR	kNN	CNN	DNN
Parameter	Tramer *et al.* [160]	ES	-	99	99	99	-	-	99
Hyper-par	Wang *et al.* [165]	ES	-	99	-	99	-	-	-
Arch.	Joon *et al.* [119]	MM	KENNEN-IO	-	-	-	-	-	88
Decision.	Papernot *et al.* [128]	SM	reservoir sampling [163]	-	-	-	-	-	84
	Papernot *et al.* [127]	SM	reservoir sampling [163]	83	61	89	85	-	89
	PRADA [84]	SM	-	-	-	-	-	-	67
Func.	Silva *et al.* [45]	SM	-	-	-	-	-	98	-
	Orekondy *et al.* [122]	SM	random, adaptive sampling	-	-	-	-	98	-

https://www.mlsecurity.ai/post/what-is-model-stealing-and-why-it-matters

Defending Against Model Stealing

Defending Against Model Stealing

It’s … hard.

- There is no known effective pure ML defense.

Defending Against Model Stealing

It’s … hard.

- There is no known effective pure ML defense.
- Existing methods:
  - Daily limit for requests -> makes it more time consuming

Defending Against Model Stealing

It’s … hard.

- There is no known effective pure ML defense.
- Existing methods:
  - Daily limit for requests -> makes it more time consuming

    - But does not solve the problem!
  - The legal system exists!
    - Let’s try to use it

The legal system

Intellectual Property

An ML model can be considered intellectual property. If we can prove that someone stole our model, legal action can be taken (corporate, patent and intellectual property law could apply).

Intellectual Property

An ML model can be considered intellectual property. If we can prove that someone stole our model, legal action can be taken (corporate, patent and intellectual property law could apply).

- How could one go at proving ownership?
  - Have some method to identify a model, even if it is a stolen copy.
  - Can also prevent misuse (deep-fakes, fake-news…) by proving outputs are from a particular model

Watermarking

Watermarking - Introduction

Goal: indicate ownership of an object.

Usual use-case: indicating copyright for images/videos by using a company logo.

What if we could do the same for DNNs?



Watermarking - Definition

Def: DNN watermarking is a method designed to detect surrogate models. Watermarking embeds a message into a model that is later extractable using a secret key. *(N. Lukas)*

Watermarking - Definition

Def: DNN watermarking is a method designed to detect surrogate models. Watermarking embeds a message into a model that is later extractable using a secret key. *(N. Lukas)*

!! Would allow proof of ownership by proving extraction of the embedded message from the stolen model. Legal action can then be taken.

Watermarking Scheme - Definition

Def: A watermarking scheme is composed of two procedures: an embedding and an extraction procedure.

Watermarking Scheme - Definition

Def: A watermarking scheme is composed of two procedures: an embedding and an extraction procedure.

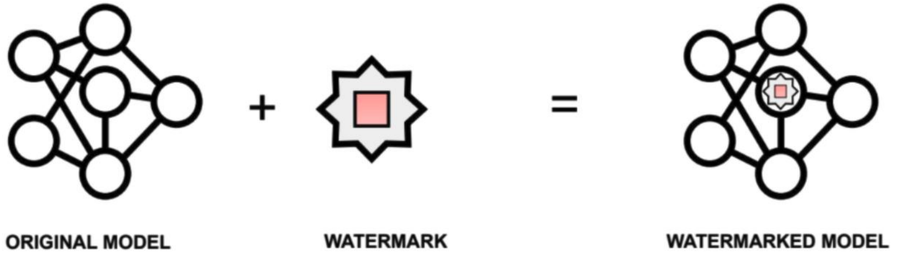
- $\mathrm{Embed}(T, m, M)$: Takes a watermarking key T, a message $m \subset \{0,1\}$ and a model M and outputs a marked model $\widehat{M}$ embedded with a message m.

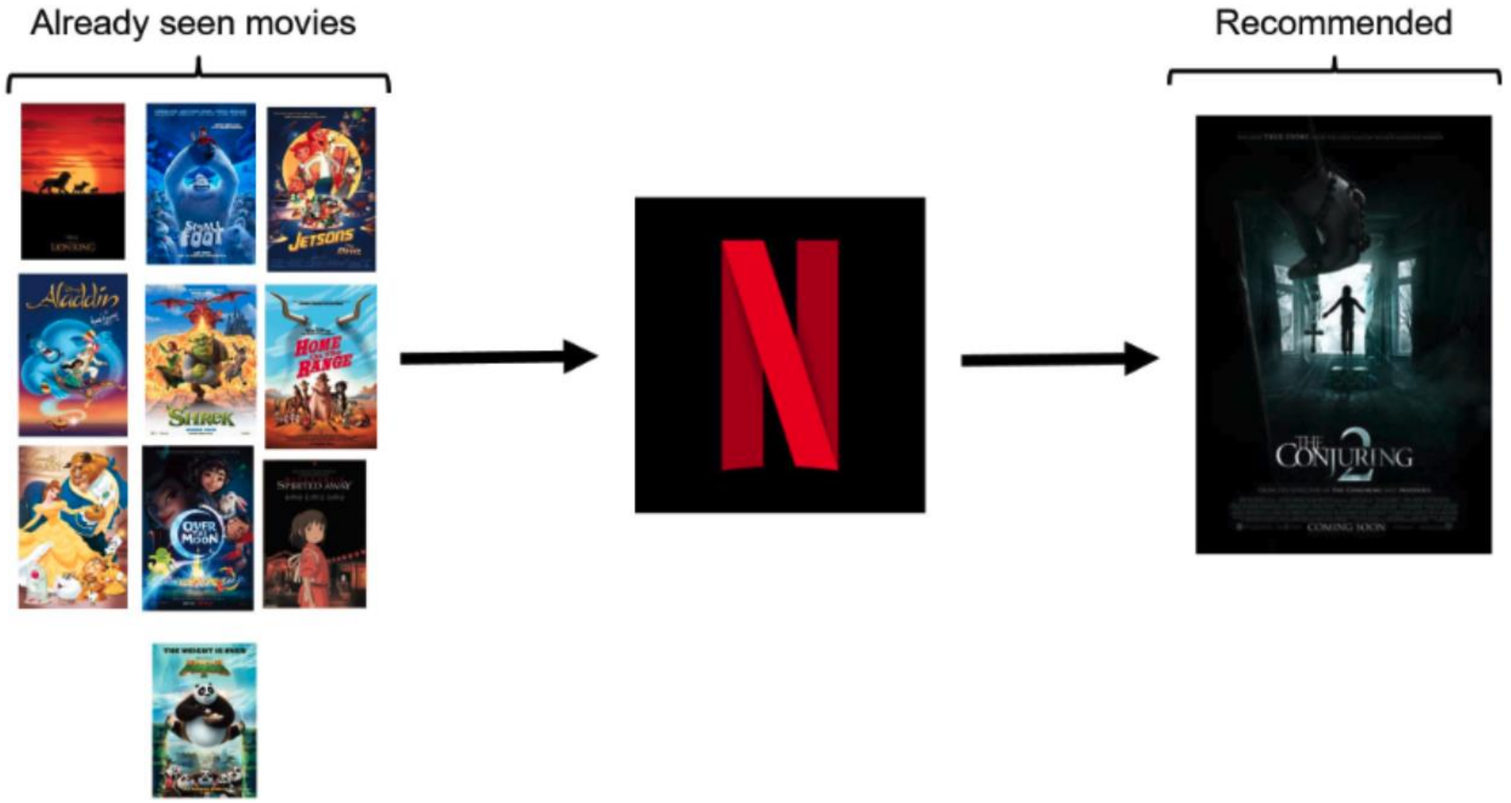
Watermarking Scheme - Definition

Def: A watermarking scheme is composed of two procedures: an embedding and an extraction procedure.

- $\mathrm{Embed}(T, m, M)$: Takes a watermarking key T, a message $m \subset \{0,1\}$ and a model M and outputs a marked $\widehat{M}$ model embedded with a message m.
- $\mathrm{Extract}(T, M)$: Takes a watermarking key T, a model M and outputs the message $m \subset \{0,1\}$ extracted from model M using key T.

Quick watermarking scheme





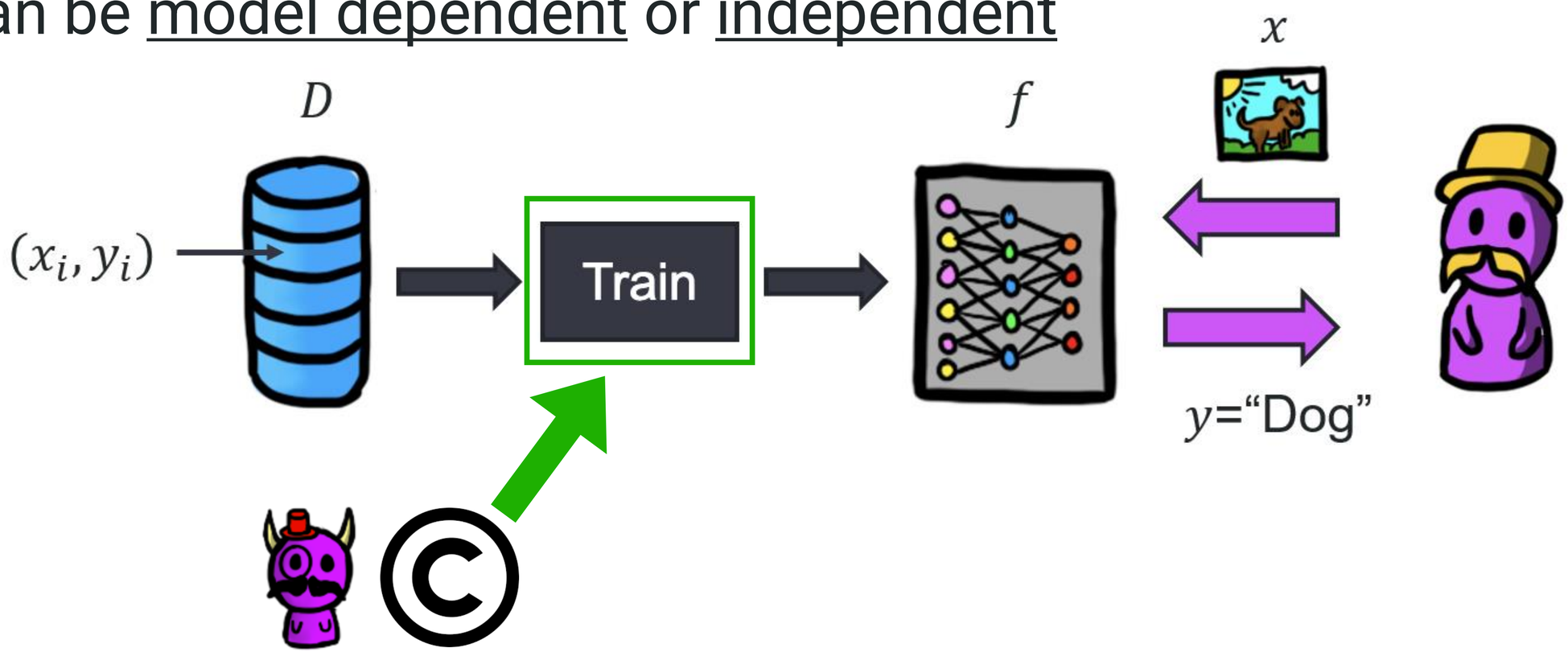
Watermarking - Ideal Requirements

Requirements	Description
Fidelity	The impact on the model's task accuracy is small.
Robustness	Surrogate models retain the watermark.
Integrity	Models trained without access to the source model do not retain the watermark.
Capacity	The watermark allows encoding large messages sizes.
Efficiency	Embedding and extracting the watermark is efficient.
Undetectability	The watermark cannot be detected efficiently without knowledge of the secret watermarking key.

Watermarking - Watermark Categories

During Training

Key can be model dependent or independent



Adi et al. 2018 – Turning Your Weakness Into a Strength

Idea: Create a watermark by assigning arbitrary labels to certain inputs
(we'll call these *triggers*)

- Take random images from outside the training distribution
- Label them randomly
- Use these inputs to train the model



Triggers

+



Original training data

Adi et al. 2018 – Turning Your Weakness Into a Strength

Idea: Create a watermark by assigning arbitrary labels to certain inputs
(we'll call these *triggers*)

- Take random images from outside the training distribution
- Label them randomly
- Use these inputs to train the model

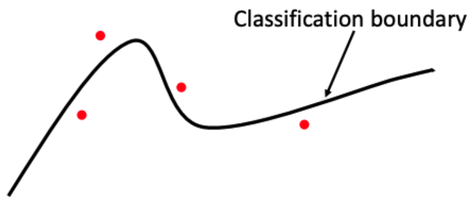
Verification: Query a model with the *trigger* input set and check whether it returns the same labels assigned before.

https://www.usenix.org/conference/usenixsecurity18/presentation/adi

Cao et al. 2021 – IPGuard

Idea: DNNs have decision boundaries shaped by their parameters. Different models will expectedly behave differently for some inputs close to the boundary

- Search for samples close to the decision boundary (we'll call them *key* inputs)
- Record their labels



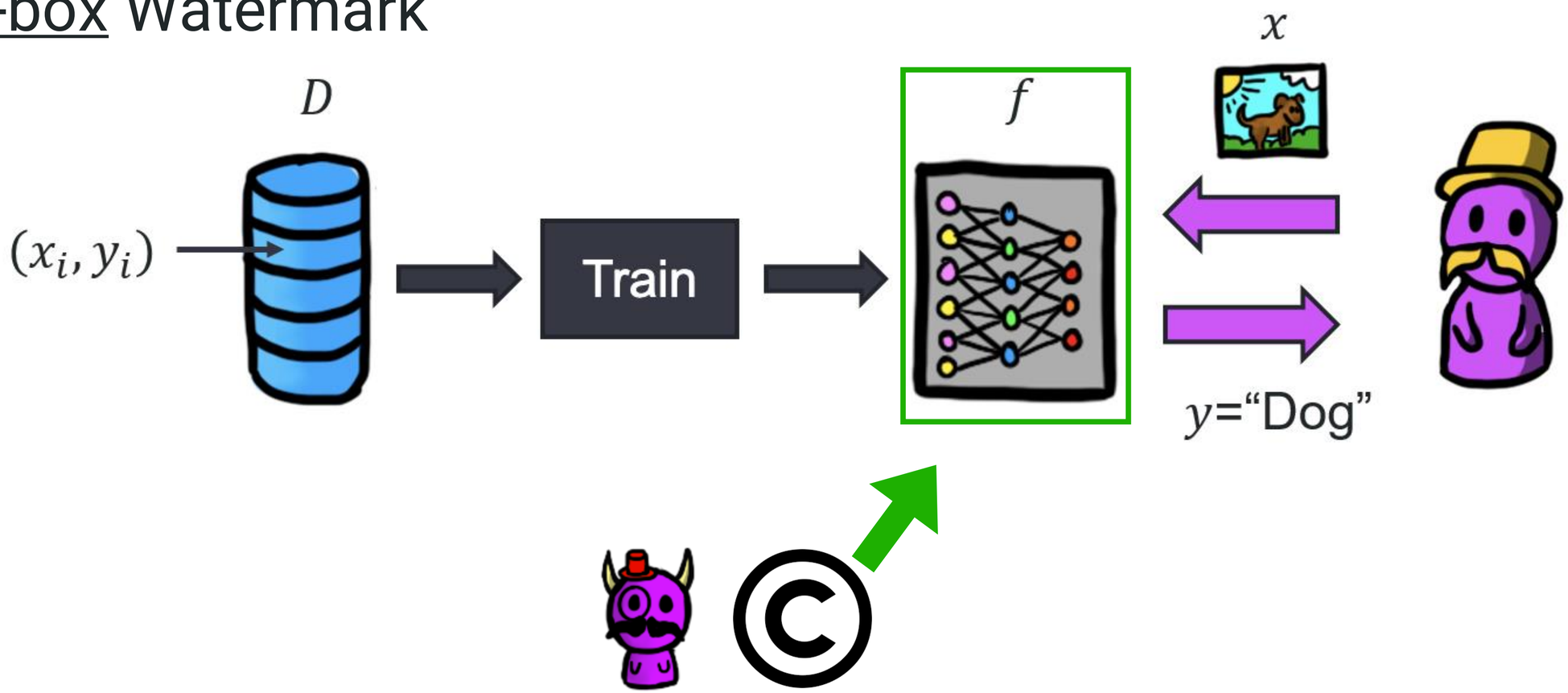
Verification: Query a model with the *key* input set and check how predictions return the same labels assigned before. Verification succeeds if a fraction of predictions (above a threshold) are correct.

https://dl.acm.org/doi/pdf/10.1145/3433210.3437526

Watermarking - Watermark Categories

After Training

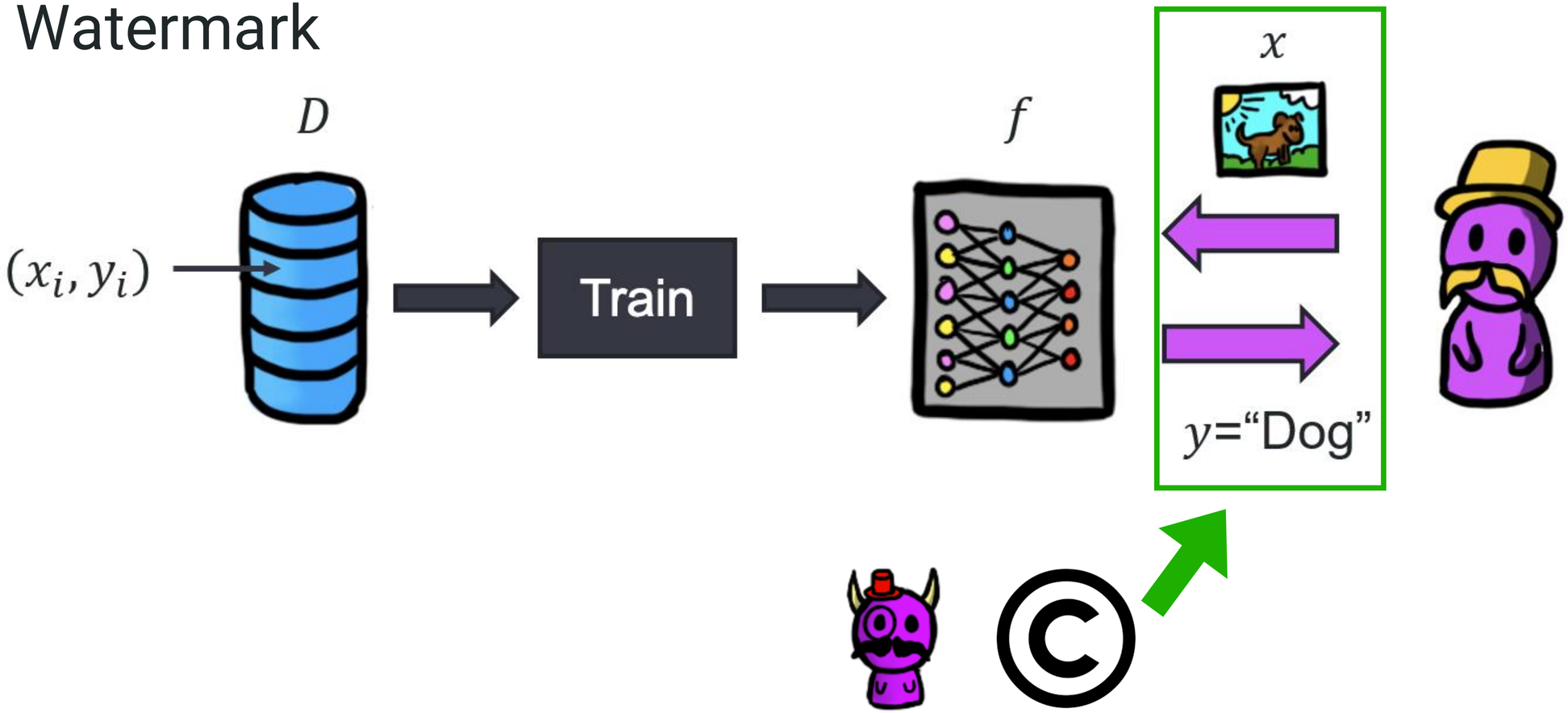
White-box Watermark



Watermarking - Watermark Categories

During Inference

Active Watermark



Watermarking - Example: DAWN

DAWN is an active multi-bit watermarking scheme. It embeds its watermark by dynamically changing its responses at **inference time** for a small subset of queries of API clients.

Watermarking - DAWN **Embed**

Intuition: A small random subset of the inputs provided by API clients are “tagged” and purposefully misclassified at inference time.

Watermarking - DAWN **Embed**

Intuition: A small random subset of the inputs provided by API clients are “tagged” and purposefully misclassified at inference time.

For an input x and model M with prediction $M(x) = y_0$, with a probability r, we output instead $y_1 \neq y_0$ and memorize the mapping $x \rightarrow y_1$.

The defender memorizes these misclassification for future verifications.

Watermarking - DAWN **Verify**

Intuition: When given an API to a potential stolen model, the verification procedure queries the API with the saved “tagged” inputs.

Watermarking - DAWN **Verify**

Intuition: When given an API to a potential stolen model, the verification procedure queries the API with the saved “tagged” inputs.

So for some model M', and all (x_i, y_i) pairs in the set of tagged inputs, we compute $e = \mathbb{E}(M'(x_i) = y_i)$. If e is greater than some threshold, we say the model was stolen.

Can an attacker remove watermarks?

Removal - Goals

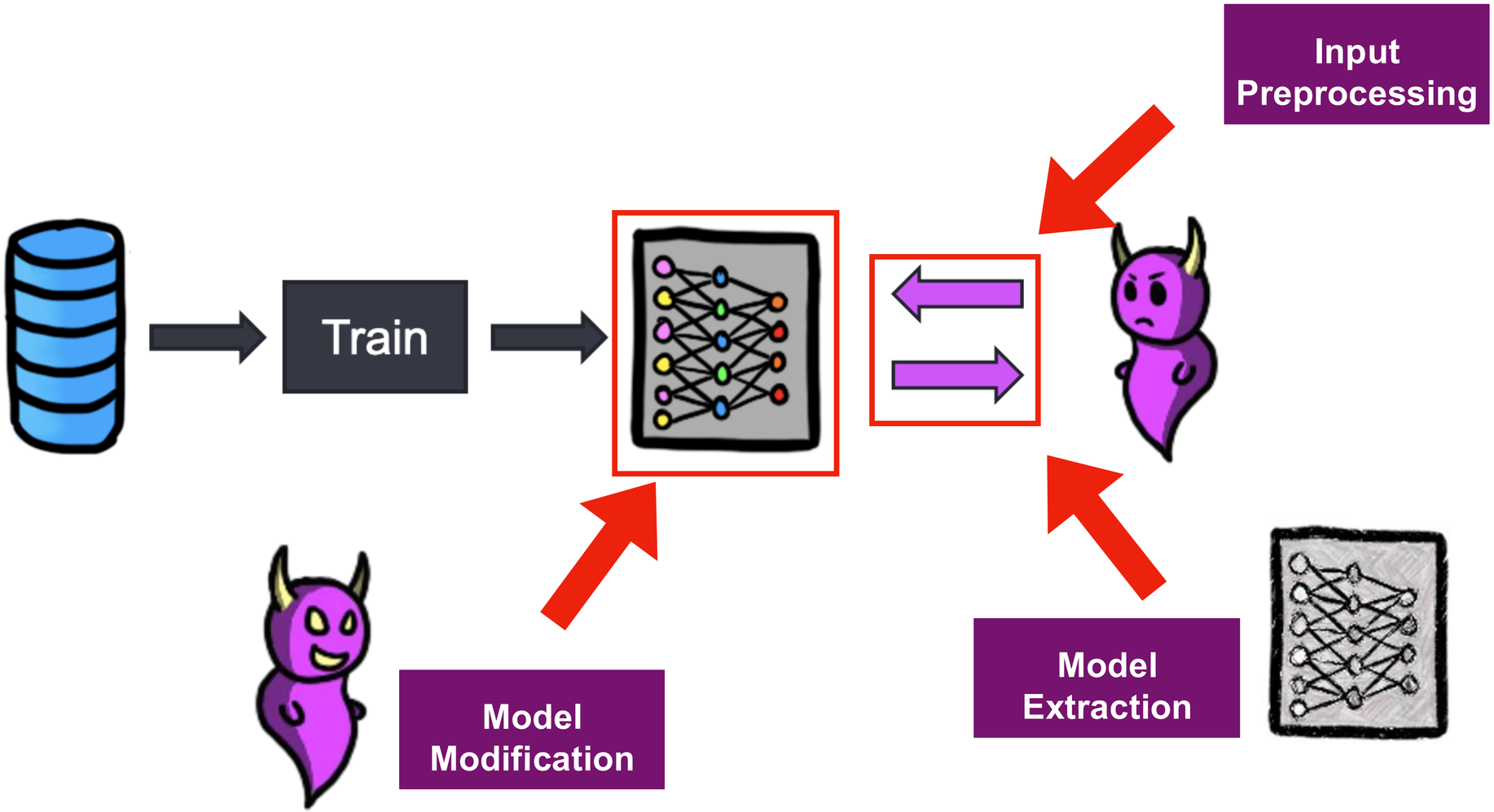
Goal 1:

The watermark needs to be removed

Goal 2:

The surrogate model needs to retain a similar test accuracy

Watermark Removal - Categories

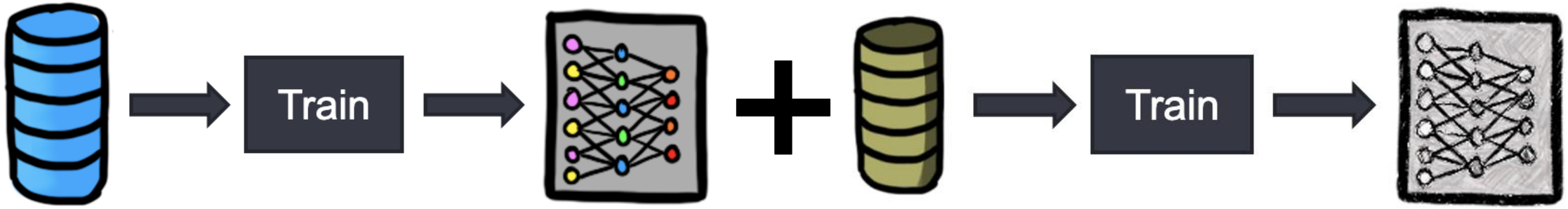


Watermark Removal - Simple Examples

Fine-tuning and Pruning are two examples of basic watermark removal.

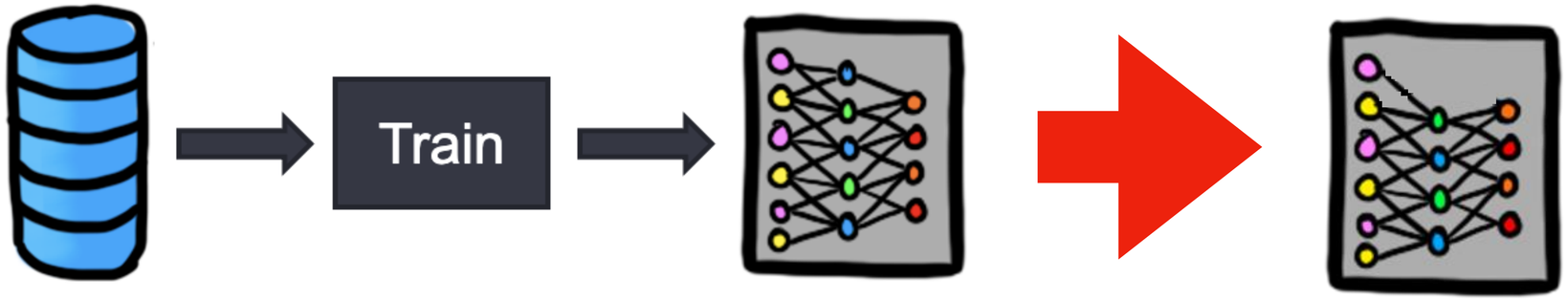
Watermark Removal - Simple Examples

Def (Fine-tuning): The process of further training a pre-trained network on a set of new inputs in the same domain (and most of the time, similar distribution).



Watermark Removal - Simple Examples

Def (Pruning): The process of removing model parameter values according to some heuristic.



Watermarking - Conclusion

Watermarking DNNs is a very active area of research.

No current watermarking scheme manages to be robust against all watermark removal attacks.



No current watermark removal attack manages to remove all watermarks.



Evasion Attacks

Poisoning vs. Evasion

- Data Poisoning attacks: Training time attack.

- Evasion Attack: Inference time attack.
  - **Q:** Why would we want to attack at inference time?

Evasion Attack - Motivations

- Evading a detection system:
  - Facial Recognition
  - Content Filter
  - Fraud Detection
- **Goal:** Lower the target model’s performance

Evasion Attack - Adversarial Examples

Input samples crafted for evasion attacks: Adversarial Examples.

Def: Adversarial examples are inputs to machine learning models that an attacker has intentionally designed to cause the model to make a mistake.

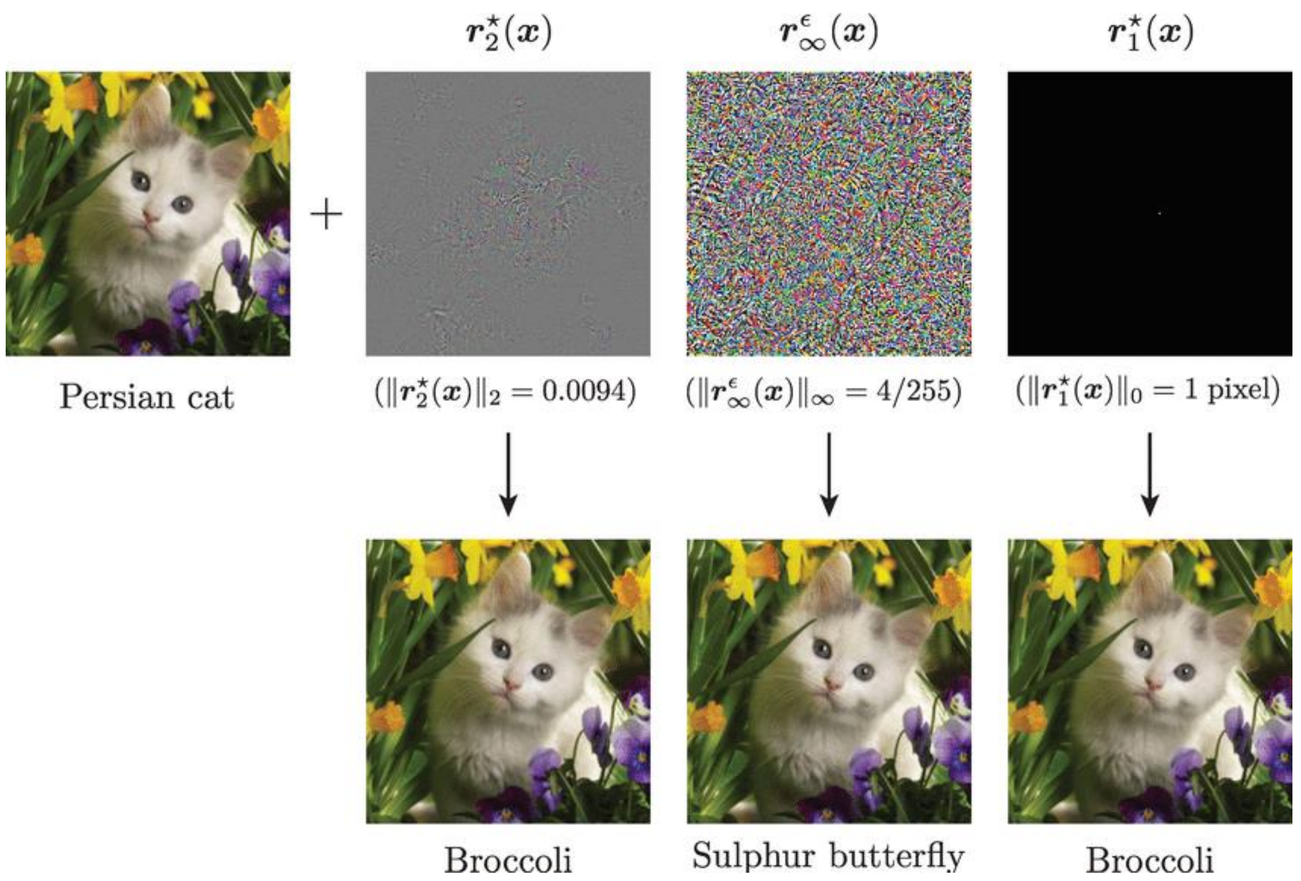
- First discovered in DNNs by *Christian Szegedy et al.* in 2014.

Adversarial Examples - Categories

Depending on the objective of the attacker, an adversarial example might have different limitations.

Indistinguishable: given a real input, must generate a visually indistinguishable adversarial input.

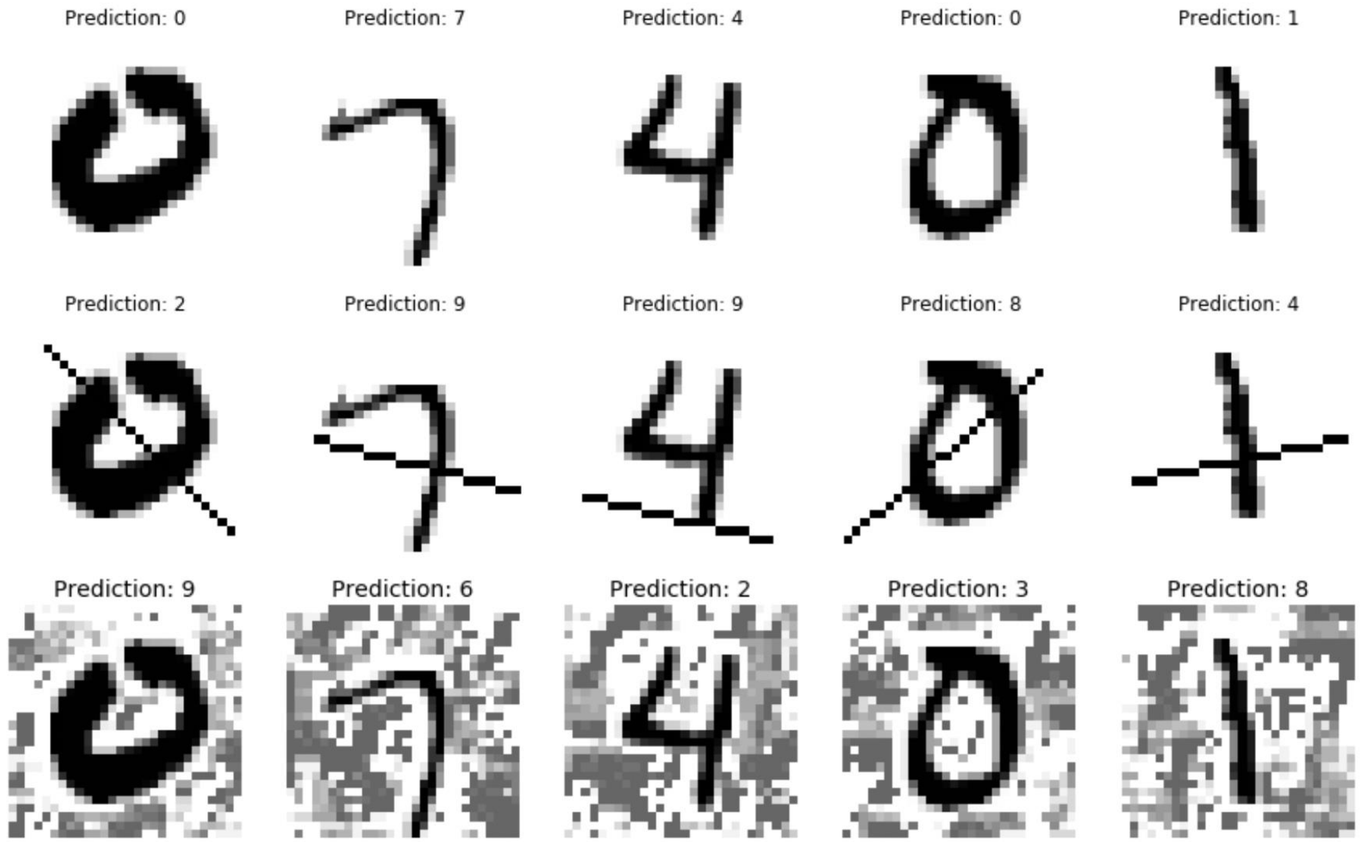
Necessary if content is human-curated.



Adversarial Examples - Categories

Content-preserving: given a real input, must generate a new input where the content is preserved.

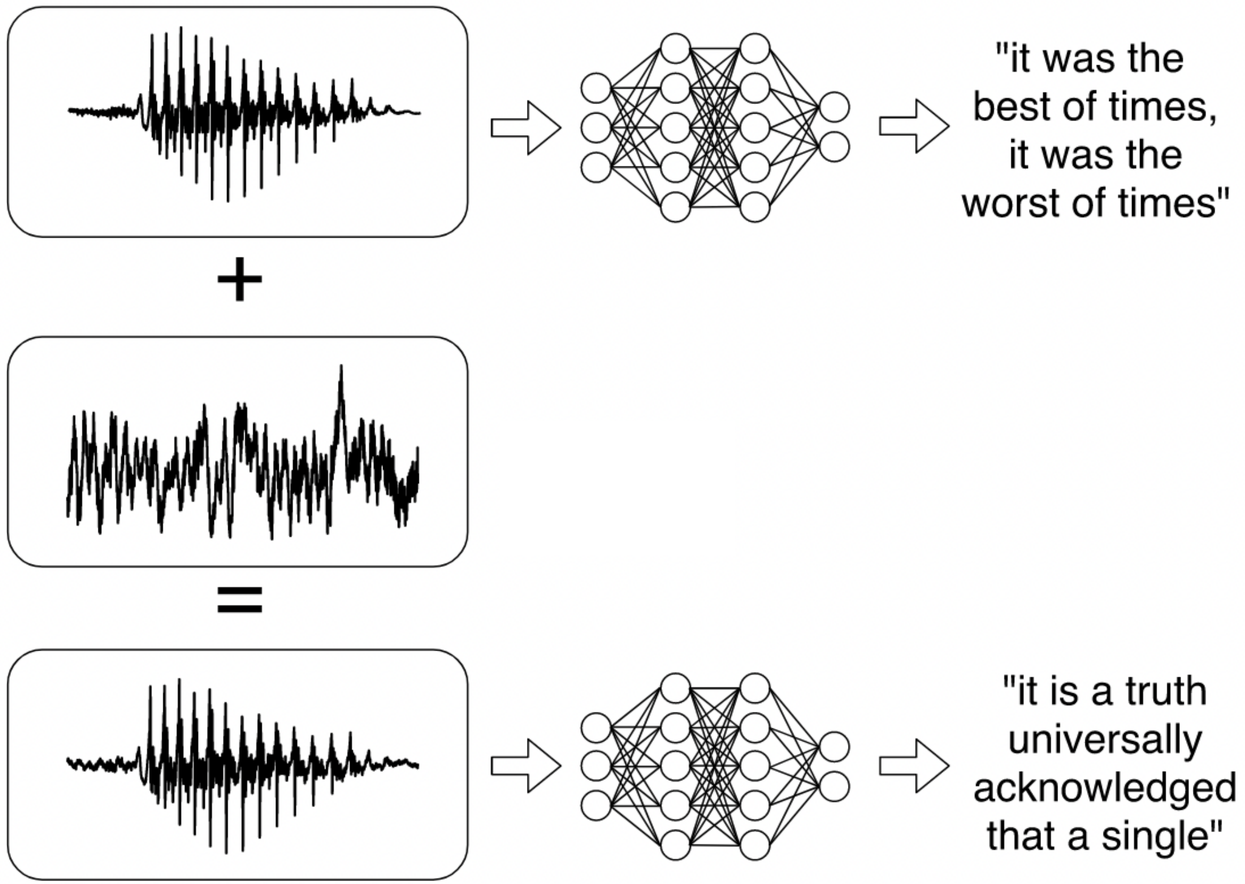
Example: re-uploading movies on Youtube with weird resizing & other effects to trick a detection algorithm



Adversarial Examples - Categories

Non-suspicious: The attacker can produce any input example they wish, as long as it would appear to a human to be a real input.

Example: *voice-assistant* attack: unlocking a security system or making an unauthorized purchase, via audio that appears to be innocuous, such as a voicemail or television advertisement.



Adversarial Examples - Categories

Content-constrained: The attacker can produce any input example they wish, as long as it contains some content payload.

Example: Email spams.



Adversarial Examples - Categories

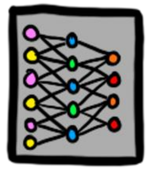
Unconstrained: The attacker can produce any input they want in order to induce desired behavior from the machine learning system.

Example: Unlocking a stolen phone by tricking fingerprint/face-recognition system

Adversarial Examples - Attack Settings

Similarly to watermarking, adversarial examples can be considered under different settings:

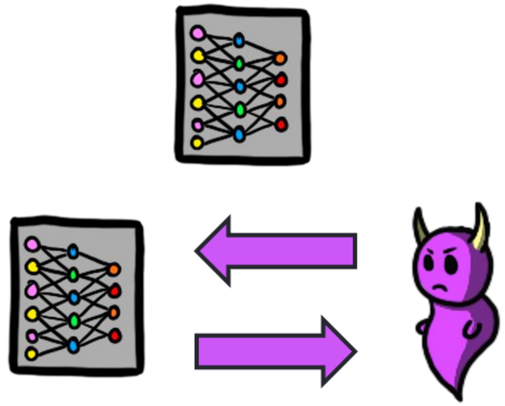
- White-box → Model is known



Adversarial Examples - Attack Settings

Similarly to watermarking, adversarial examples can be considered under different settings:

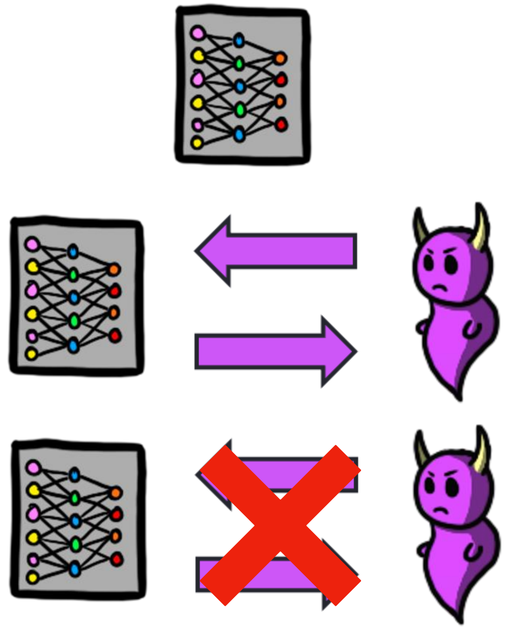
- White-box → Model is known
- Black-box → Query access to the model



Adversarial Examples - Attack Settings

Similarly to watermarking, adversarial examples can be considered under different settings:

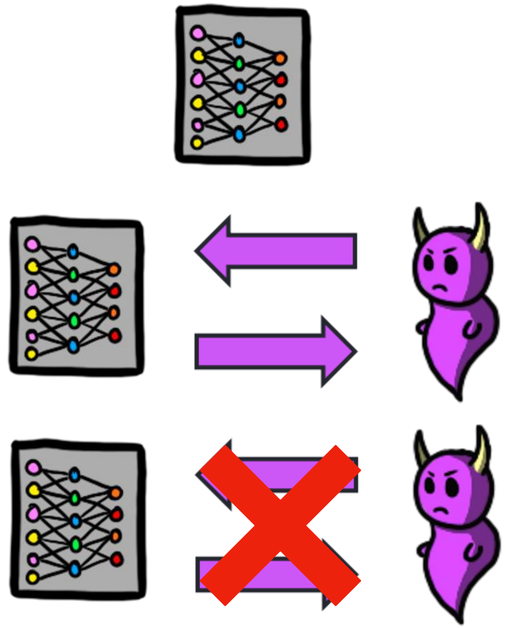
- White-box → Model is known
- Black-box → Query access to the model
- Transferable (to another model) → No query access



Adversarial Examples - Attack Settings

Similarly to watermarking, adversarial examples can be considered under different settings:

- White-box → Model is known
- Black-box → Query access to the model
- Transferable (to another model) → No query access
- Gray-box → The rest



Adversarial Examples - Defenses

Similarly to many ML-related problems, there is no existing defense that can fully prevent adversarial examples.

What properties do we want from a defense?

- It preserves clean input accuracy.
- It correctly classifies adversarial examples

Adversarial Examples - Defenses

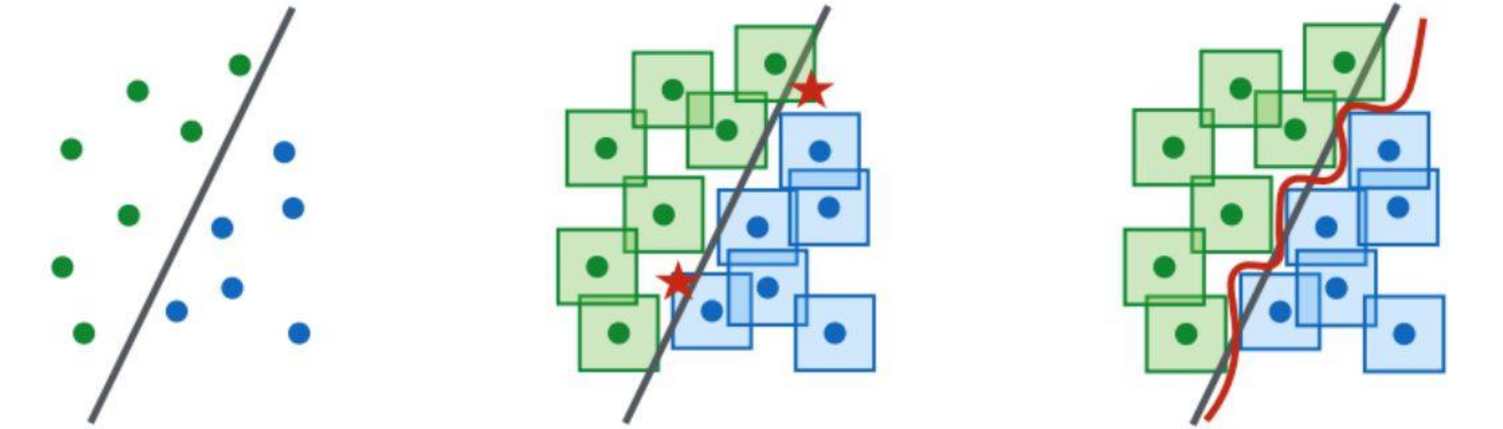
Any guesses as to how we could go about defending against adversarial examples?

Basic Defense - Adversarial Training

Adversarial Training is a simple defense that goes as follows:

- For a batch D_i of input samples$D_i = \{(x_1, y_1), (x_2, y_2), \ldots, (x_b, y_b)\}$, b is the batch size.
- Generate adversarial examples $D_i' = \{(x_1', y_1), (x_2', y_2), \ldots, (x_b', y_b)\}$
- Train your model on $\overline{D_i} = D_i \cup D_i'$

Basic Defense - Adversarial Training



(a) Linearly Separable Samples

(b) Samples Augmented with Adversarial Examples

(c) Complex Decision Boundary

Augmenting Training Data with Adversarial Examples

Basic Defense - Adversarial Training

Adversarial Training is simple, but effective. It is currently considered one of if not the best existing defense against adversarial examples by the research community.